

Auth & Session Security Analyzer

Rapport d'audit de securite Android

Application analysee : app-debug
Type d'authentification : SESSION_COOKIE
Vulnerabilites detectees : 46
Date d'analyse : 14/05/2026 23:21

Score global de securite : 0 / 10 (CRITIQUE)

1. Resume Executif

Ce resume presente les principales vulnerabilites identifiees lors de l'audit, evalue le niveau de risque global et formule les recommandations prioritaires.

L'application analysée est InsecureBankv2, une application Android de démonstration simulant une application bancaire mobile utilisant un mécanisme d'authentification par cookie de session (SESSION_COOKIE). L'audit de sécurité conduit selon le référentiel OWASP MASVS v2, combinant analyses statique et dynamique, révèle un niveau de risque global évalué comme ****critique****, avec des scores particulièrement alarmants dans les domaines réseau (0/10) et stockage (2/10). Les vulnérabilités les plus graves incluent la transmission des credentials en clair sur HTTP sans aucun chiffrement TLS, l'absence totale de gestion de session côté serveur (aucun cookie JSESSIONID émis, retransmission des identifiants à chaque requête), des secrets et mots de passe codés en dur dans le code source, ainsi que la journalisation des credentials utilisateur dans les logs Android accessibles à toute application tierce disposant de la permission adéquate. L'impact potentiel pour les utilisateurs est majeur : un attaquant positionné sur le réseau peut intercepter sans effort les identifiants bancaires, usurper des sessions, accéder aux comptes et effectuer des opérations frauduleuses, sans nécessiter de compétences techniques avancées. Les trois recommandations prioritaires sont les suivantes : premièrement, migrer immédiatement l'intégralité des communications vers HTTPS avec validation stricte du certificat TLS et déploiement d'une Network Security Configuration interdisant le trafic HTTP en clair ; deuxièmement, implémenter une gestion de session conforme aux standards (émission d'un cookie de session sécurisé avec les attributs Secure, HttpOnly et SameSite, invalidation serveur à la déconnexion, timeout d'inactivité) ; troisièmement, supprimer toutes les occurrences de credentials codés en dur et désactiver toute journalisation d'informations sensibles avant toute mise en production de l'application.

2. Analyse Statique - Vulnerabilites detectees

L'analyse statique consiste en la decompilation de l'APK avec JADX et l'examen des sources Java/Kotlin par patterns regex. Les resultats sont classes par severite : **CRITIQUE** (-3 pts), **ELEVE** (-2 pts), **MOYEN** (-1 pt).

Fichiers analyses : 2918 | Vulnerabilites : 42 | Score : 0/10

Severite	Type	Fichier	Detail
CRITIQUE	Secret code en dur	AccessibilityNodeInfoCo...	password: "); builder.append(isPassword()); builder.app...
CRITIQUE	Secret code en dur	ChangePassword.java	password=" + this.uname); this.textView_Username = (Text...
ELEVE	Identifiants dans les logs	DoLogin.java	Log.d("Successful Login:", " , account=" + DoLogin.this.username ...
ELEVE	Hachage MD5 faible	zza.java	MessageDigest.getInstance("MD5"
MOYEN	Communication en clair (HTTP)	AnalyticsReceiver.java	http://g
MOYEN	Communication en clair (HTTP)	CampaignTrackingRecei...	http://g
MOYEN	Communication en clair (HTTP)	CampaignTrackingRecei...	http://g
MOYEN	Communication en clair (HTTP)	Tracker.java	http://h
MOYEN	Communication en clair (HTTP)	zza.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g

MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzy.java	http://w
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	zzm.java	http://p
ELEVE	Hachage MD5 faible	zzak.java	MessageDigest.getInstance("MD5")
ELEVE	Hachage MD5 faible	zzbl.java	MessageDigest.getInstance("MD5")
MOYEN	Communication en clair (HTTP)	zzgk.java	http://w
ELEVE	Hachage MD5 faible	zzhl.java	MessageDigest.getInstance("MD5")
MOYEN	Communication en clair (HTTP)	PlusOneButton.java	http://s
MOYEN	Communication en clair (HTTP)	PlusOneButton.java	http://s
MOYEN	Communication en clair (HTTP)	zzax.java	http://h

3. Analyse Dynamique - Tests d'authentification

L'analyse dynamique évalue le comportement de l'application en cours d'exécution. Les tests portent sur la validité des tokens après déconnexion, la durée de vie, la rotation des tokens et la robustesse du mécanisme d'authentification.

Mode d'analyse : API_DIRECT

Déconnexion effective : Non (VULNERABLE - rejeu possible)

Durée de vie du token : None min

Rotation des refresh tokens : Non testé

Vulnérabilités détectées lors des tests :

CRITIQUE	POST http://localhost:8888/login envoie username+password sans chiffrement - interceptable pa
ELEVE	InsecureBankv2 n'emet aucun token/cookie de session. Les credentials sont retransmis a chaque r
ELEVE	10 tentatives de login echouees consecutives acceptees sans blocage ni CAPTCHA
MOYEN	L'application n'a pas de mecanisme de logout cote serveur. Le menu 'Restart' ne fait que navigu

4. Scores MASVS par Domaine

Les scores reflètent le niveau de sécurité par domaine OWASP MASVS v2. Chaque vulnérabilité déduit des points selon sa sévérité. Un score inférieur à 4 indique un risque critique sur ce domaine.

MASVS-AUTH		6/10
MASVS-STORAGE		2/10
MASVS-CRYPTO		2/10
MASVS-NETWORK		0/10

5. Checklist de Sécurité (MASVS)

Checklist générée par IA (Claude) adaptée au type d'authentification détecté. Chaque contrôle est mappe à un chapitre OWASP MASVS pertinent. Ces points constituent le référentiel de vérification pour la revue de code.

- [] [MASVS-AUTH-1] Vérifier que les credentials utilisateur (nom d'utilisateur et mot de passe) sont transmis exclusivement via HTTPS/TLS et jamais en clair sur HTTP
- [] [MASVS-AUTH-2] Vérifier que l'application émet et gère un cookie de session valide (JSESSIONID) après authentification, sans retransmettre les credentials à chaque requête
- [] [MASVS-AUTH-3] Vérifier que le cookie de session est configuré avec les attributs Secure, HttpOnly et SameSite afin de prévenir l'interception et les attaques XSS/CSRF
- [] [MASVS-AUTH-4] Vérifier que le serveur invalide effectivement la session côté serveur lors de la déconnexion de l'utilisateur, et non pas seulement côté client
- [] [MASVS-AUTH-5] Vérifier qu'un mécanisme de protection contre les attaques par force brute est en place (verrouillage de compte, délai exponentiel ou CAPTCHA après N tentatives échouées)
- [] [MASVS-AUTH-6] Vérifier que les sessions disposent d'une durée de vie maximale et d'un timeout d'inactivité configurés et appliqués côté serveur
- [] [MASVS-AUTH-7] Vérifier que les tokens/cookies de session sont régénérés après chaque authentification réussie afin de prévenir les attaques de fixation de session
- [] [MASVS-STORAGE-1] Vérifier qu'aucun credential (mot de passe, identifiant) n'est stocké en clair dans le code source, les logs ou les fichiers de configuration de l'application
- [] [MASVS-STORAGE-2] Vérifier que les cookies de session ne sont pas persistés dans un stockage non sécurisé (SharedPreferences non chiffrés, base de données SQLite sans protection, logs système)
- [] [MASVS-NETWORK-1] Vérifier qu'une Network Security Configuration (NSC) est définie et interdit explicitement tout trafic HTTP en clair pour l'ensemble des domaines applicatifs
- [] [MASVS-NETWORK-2] Vérifier que le certificat TLS du serveur est validé correctement par l'application et qu'aucun mécanisme de contournement de validation (acceptation de tous les certificats) n'est présent
- [] [MASVS-CRYPTO-1] Vérifier que l'application n'utilise pas d'algorithmes de hachage faibles ou obsolètes (MD5, SHA-1) pour toute opération liée à la sécurité (intégrité, authentification, stockage de mots de passe)

- [] [MASVS-CRYPTO-2] Vérifier qu'aucune clé, secret ou mot de passe n'est codé en dur dans le code source ou dans les ressources compilées de l'application
- [] [MASVS-CODE-1] Vérifier qu'aucune information sensible (credentials, tokens de session, données personnelles) n'est écrite dans les logs système Android (Log.d, Log.i, Log.e, etc.) en environnement de production

6. Criteres d'Acceptation de Securite

Les criteres d'acceptation definissent les conditions que le systeme doit satisfaire pour etre considere comme securise. Ils servent de base aux tests d'acceptation, aux revues de code et aux audits de conformite MASVS.

1. Le système DOIT transmettre l'ensemble des données d'authentification (identifiant, mot de passe, cookie de session) exclusivement via des canaux chiffrés TLS/HTTPS afin de prévenir toute interception par attaque Man-in-the-Middle.
2. Le système NE DOIT PAS retransmettre les credentials (nom d'utilisateur et mot de passe) à chaque requête HTTP afin d'éviter l'exposition répétée de données sensibles sur le réseau et de respecter le principe de session stateful.
3. Le système DOIT invalider le cookie de session côté serveur lors de toute opération de déconnexion afin de garantir qu'une session terminée ne puisse être réutilisée par un attaquant ayant capturé le cookie.
4. Le système DOIT bloquer ou ralentir de manière significative l'accès au compte après un maximum de 5 tentatives d'authentification échouées consécutives afin de prévenir les attaques par force brute sur les comptes utilisateurs.
5. Le système NE DOIT PAS écrire d'informations sensibles telles que les identifiants, mots de passe ou tokens de session dans les journaux système Android (Logcat) afin d'éviter leur extraction par des applications tierces ou lors d'une analyse forensique de l'appareil.
6. Le système NE DOIT PAS utiliser l'algorithme MD5 ou tout autre algorithme cryptographique obsolète pour des opérations liées à la sécurité afin de garantir l'intégrité et la confidentialité des données protégées.
7. Le système DOIT régénérer un nouveau cookie de session immédiatement après chaque authentification réussie afin de prévenir les attaques de fixation de session (Session Fixation).
8. Le système DOIT configurer les cookies de session avec les attributs Secure, HttpOnly et une politique SameSite stricte afin de réduire la surface d'attaque liée au vol de session par interception réseau ou scripts malveillants.
9. Le système NE DOIT PAS contenir de secrets, mots de passe ou clés cryptographiques codés en dur dans le code source ou les ressources de l'application afin d'éviter leur extraction lors d'une rétro-ingénierie du binaire APK.